BuzzMe — Technical Overview

Version 1.0 · October 2026 · For team review

1. Project overview

BuzzMe is a mobile-friendly platform for meeting random strangers. Its signature feature is **random video chat**. Voice-only and text-only chat are lighter modes of the same product.

Users join as guests (accounts come later), pick a nickname, choose a mode, get matched with a stranger, and can press **Next** to meet someone new, **Stop** to leave, or **Report** to flag abuse. Security and IP protection between strangers are core requirements, not extras.

2. Recommended stack

Layer	Technology	Purpose
Backend	Node.js + Socket.IO (Express for HTTP)	Matching, sessions, text relay, WebRTC signaling, moderation APIs
Database	MySQL (PostgreSQL is an acceptable alternative)	Bans, reports, later accounts and preferences
Temporary state	In-memory in v1; Redis when running more than one server instance	Waiting queues, active sessions, rate limits
Real-time channel	WebSocket (Socket.IO)	Matching events, text chat, signaling
Audio and video	WebRTC	Low-latency media between users
Media relay	coturn (TURN), forced relay	Relays media so strangers never see each other's IPs
Frontend	Not decided (web, Flutter, React Native or other)	The backend contract is client-agnostic

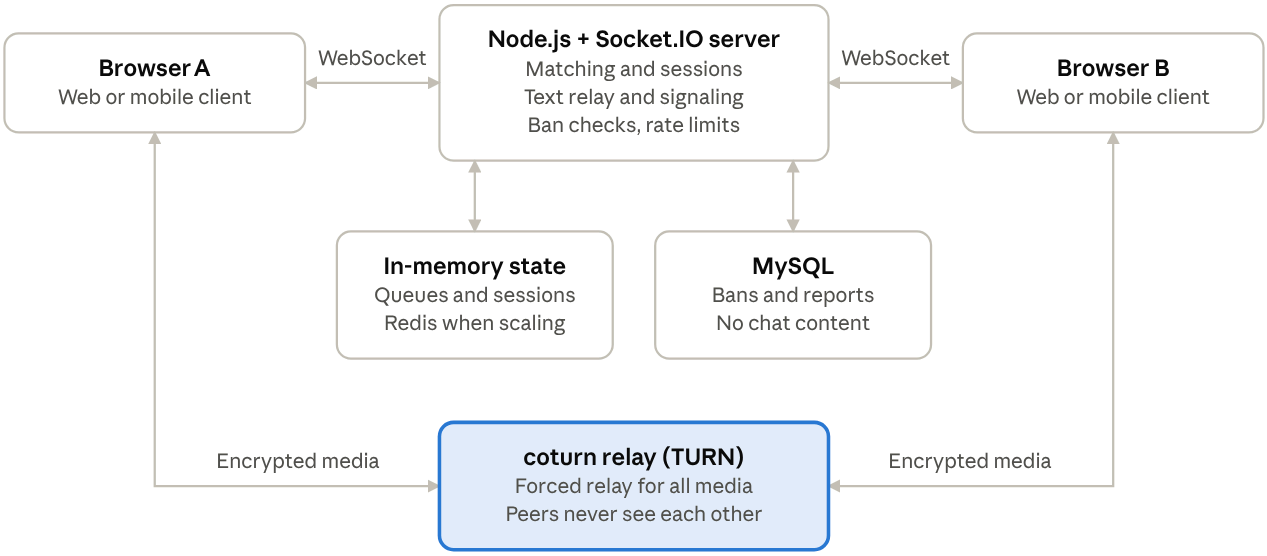
Layer	Technology	Purpose
Deployment	Linux VPS, Nginx with TLS	Node service, coturn, database

Rust was evaluated and remains an option for the matchmaker later if load proves it necessary. The first version favors speed of delivery and maintainability.

3. System architecture

Clients connect to the Node server over secure WebSocket for matching, chat and signaling. Audio and video travel between the two clients through a coturn relay, never directly. The server keeps queues and live sessions in memory and uses the database only for bans and reports.

Media flows through the relay, so peer IPs stay hidden

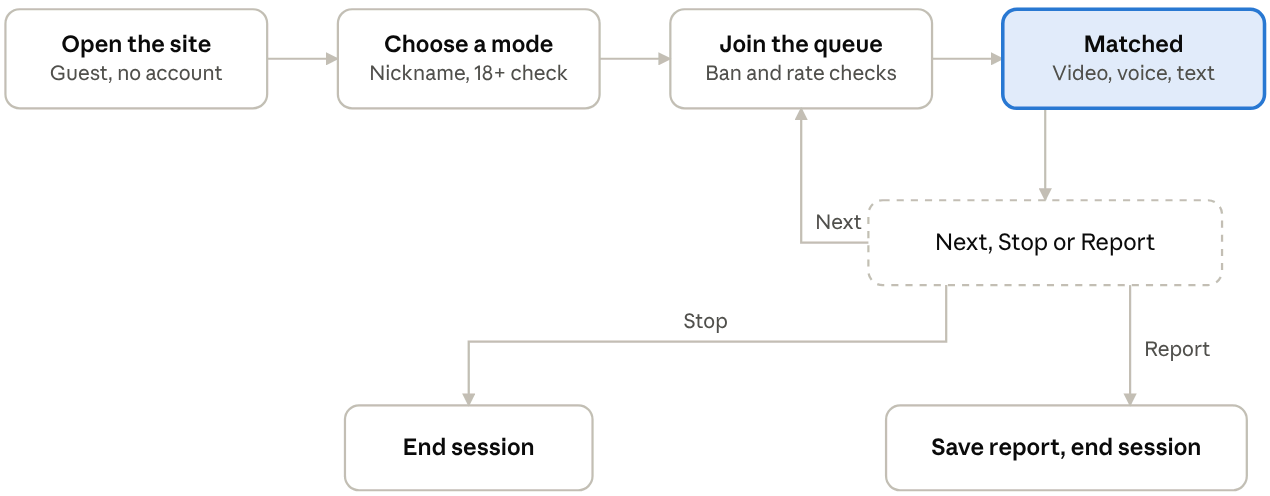


system architecture · 6 components

The highlighted relay carries all audio and video; the Node server only handles matching, chat and signaling.

4. Core user flow

A user opens the app, chooses a nickname and mode, and joins the queue. When matched, they chat by text, voice or video. From there they can press Next (back to the queue), Stop (end the session) or Report (save a report and end the session).



core user flow · 4 steps, 1 decision

Next sends the user back to the queue for a new match; Stop and Report both close the session.

5. Session modes

One matching and session system serves every mode. Each session has a mode that decides which capabilities the client enables.

Mode	Camera	Microphone	Text chat
Video (default)	Yes	Yes	Yes
Voice	No	Yes	Yes
Text	No	No	Yes

Users are only matched with others in the same mode.

6. Real-time event contract

Client to server	Payload
`join`	nickname, mode

Client to server	Payload
`next`, `stop`	none
`message`	text
`signal`	type (offer, answer, ice), data
`report`	reason (also ends the session)

Server to client	Payload
`waiting`	none
`matched`	partner nickname, initiator flag, temporary TURN credentials
`message`	text
`signal`	relayed as-is after filtering
`partner_left`	none
`error`	code (banned, rate_limited, invalid)
`online`	count

7. Data and privacy rules

Data	Stored?	Notes
Chat messages	No	In memory only; never in logs, analytics or backups
Audio and video	No	Never recorded
Waiting queue and active sessions	Temporarily	In memory
Reports	Yes	Include a short message snapshot only when a user files a report
Bans	Yes	Keyed hash of IP plus a device token

Data	Stored?	Notes
IP addresses	Hashed only	HMAC with a server secret; documented retention limit
Accounts and preferences	Later phase	Guests need no account

8. Security and IP protection

IP protection between strangers

- Clients use `iceTransportPolicy: "relay"`, so only the relay's address is ever offered to the other user.
- The server filters relayed signaling and drops `host` and `srflx` ICE candidates as a second layer.
- TURN credentials are short-lived and issued per session (coturn time-limited HMAC credentials).
- coturn blocks relaying to private IP ranges, sets per-user quotas, and offers TURN over TLS on port 443.
- Users never receive another user's IP, internal ID or account details. Sessions use random IDs.

Application hardening

- HTTPS and WSS only, with HSTS, a strict CSP and an Origin check on WebSocket upgrades.
- Every message is validated against a schema, with size caps on messages and signaling payloads.
- Chat text is rendered as plain text to prevent XSS.
- Heartbeats, idle timeouts, a connection cap per IP, and rate limits on Start, Next and messages.

User safety

- Ban check on every join. Report button available at all times and ends the session immediately.
- 18+ confirmation before the first match, nickname filtering, and optional phone verification later to deter ban evasion.

Known limits

- Media is end-to-end encrypted between browsers, so the server cannot scan video. Moderation relies on reports, bans and verification unless an SFU is adopted later.
- The relay operator can see IPs, so the platform must never promise absolute anonymity.
- Forced relay means all video bandwidth flows through coturn. This is the main running cost.

9. Data model (first version)

Table	Key fields
`bans`	id, ip_hash, device_token, reason, expires_at, created_at
`reports`	id, reporter_ip_hash, reported_ip_hash, mode, reason, message_snapshot (JSON, text mode only), created_at

10. Development phases

1. **Foundation:** database schema, Node server scaffold, config, rate limiting.
2. **Text MVP:** queue, matching, Next/Stop, message relay, ban check, report flow.
3. **Voice and video:** WebRTC signaling, forced TURN relay, mute and camera controls, reconnect handling.
4. **Safety and accounts:** admin dashboard, temporary bans, optional accounts, blocks, verification.
5. **Scale:** Redis-backed coordination, load testing, monitoring, multiple instances.

11. Open decisions

- Frontend technology (web, Flutter, React Native or other).
- MySQL or PostgreSQL for the primary database.
- Age and phone verification approach.
- Hosting provider and region for the Node service and coturn.
- Retention periods for reports and hashed IP data.